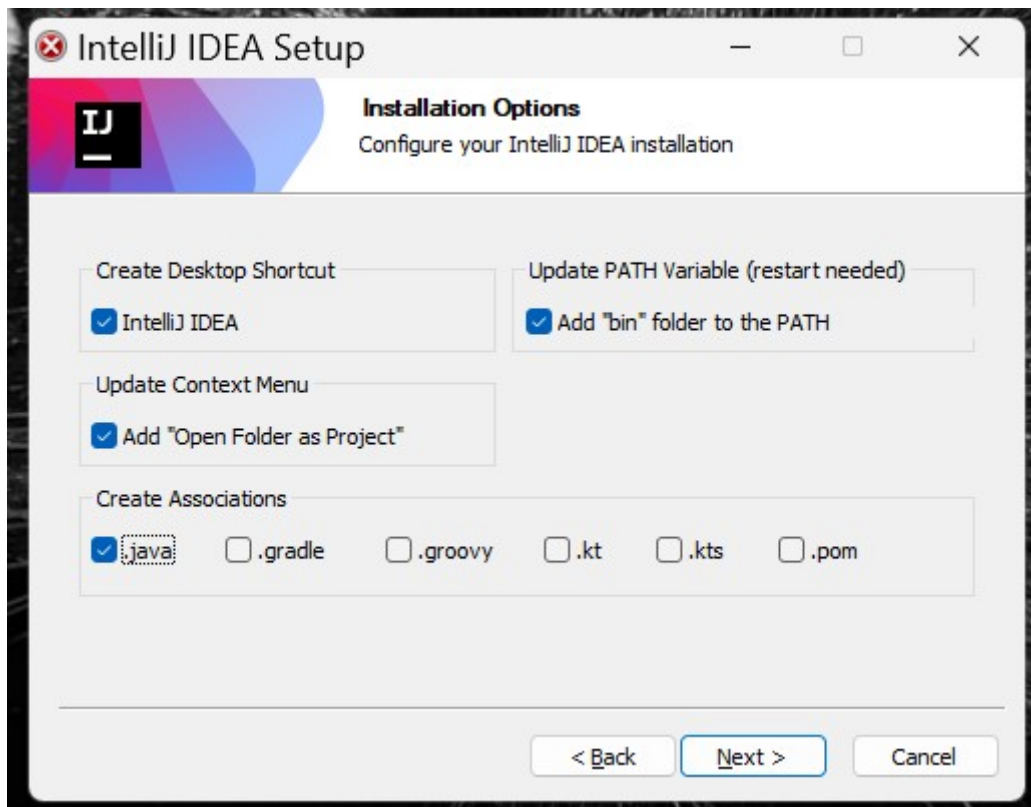


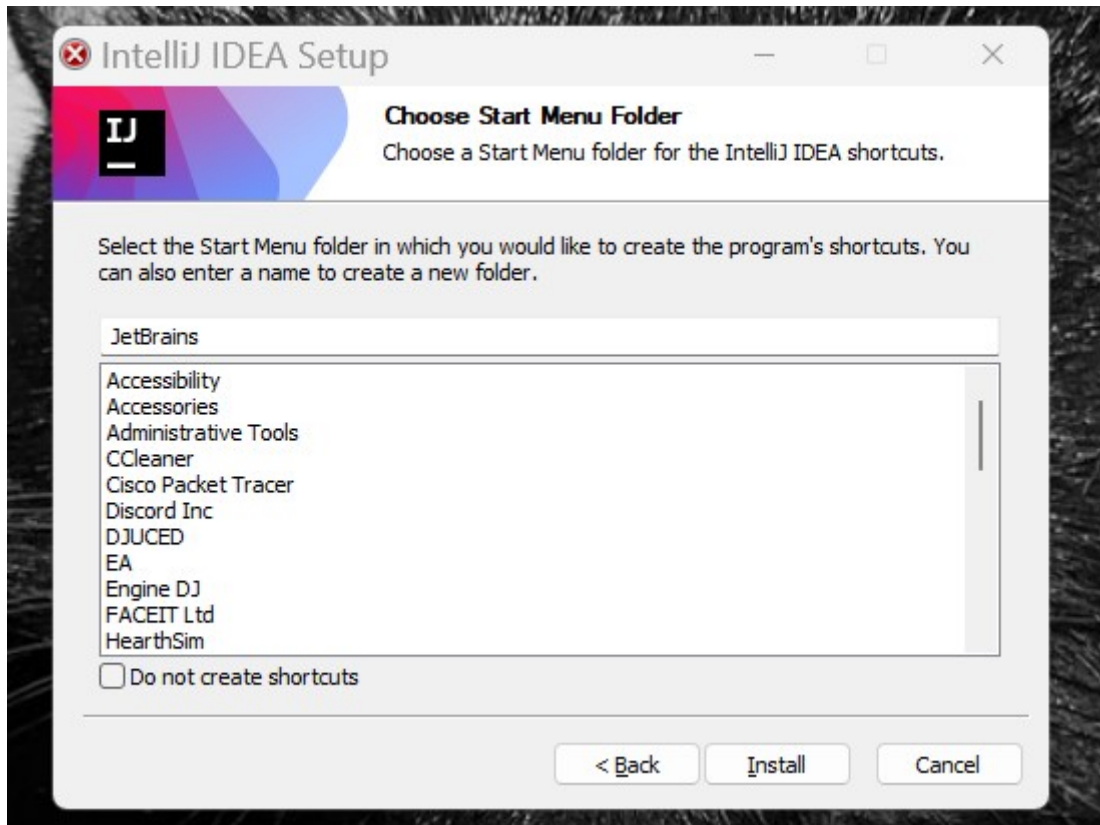
José Antonio Pastor Rivera- RA2

## Fase de Instalación (IDE Propietario: IntelliJ IDEA)



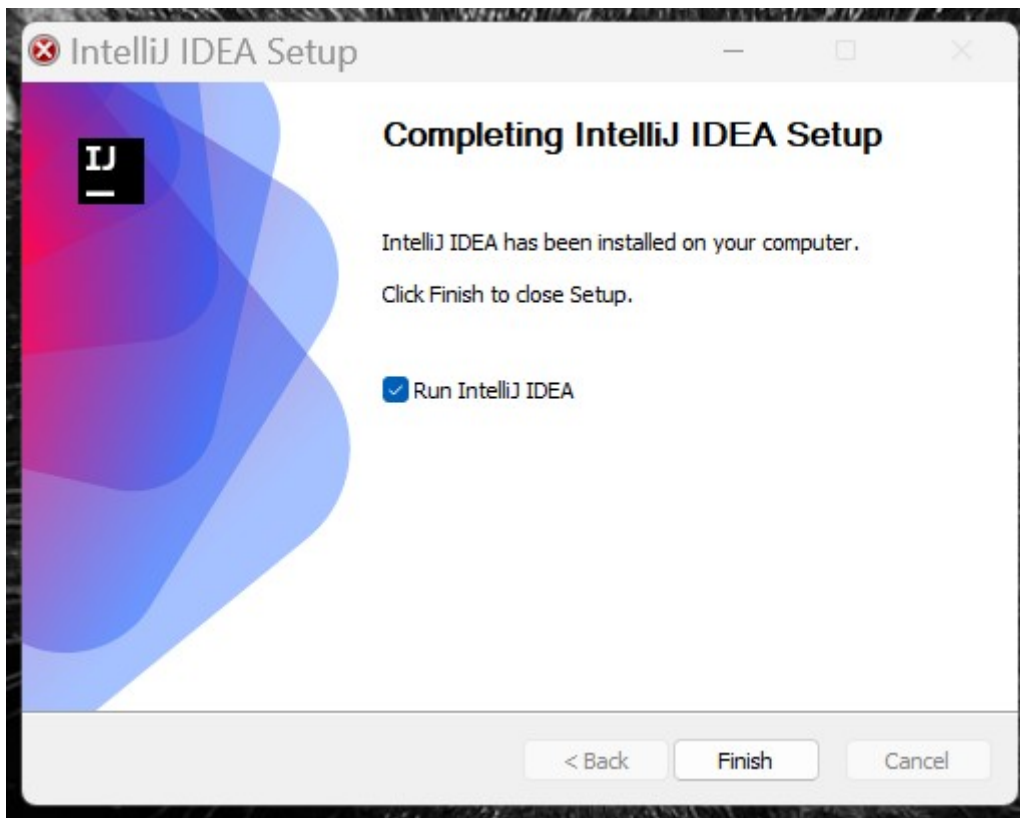
**Imagen 1:** 1.png (Opciones de instalación)

**Configuración de Instalación:** Durante el proceso de instalación, he personalizado las opciones para maximizar la integración con el sistema. He habilitado "**Add 'Open Folder as Project'**" en el menú contextual, lo cual es fundamental para la interoperabilidad futura, y he actualizado la variable **PATH** para permitir la ejecución desde consola. Asimismo, he asociado los archivos .java al entorno para asegurar su apertura predeterminada.



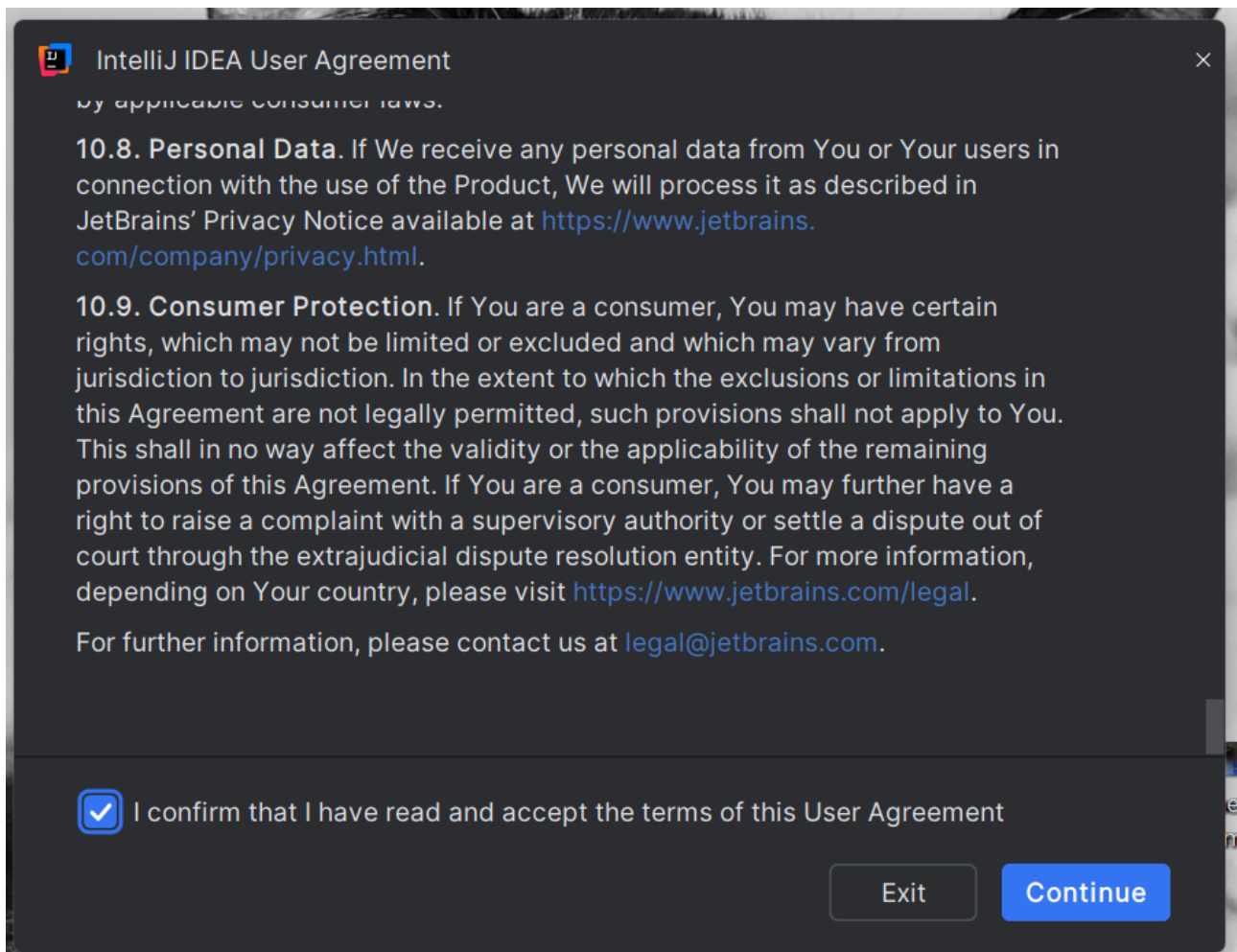
**Imagen:** 2.png (Carpeta Menú Inicio)

**Organización en el Sistema Operativo:** Durante el asistente de instalación, se ha configurado la ubicación de los accesos directos. He mantenido la selección predeterminada para crear la carpeta "JetBrains" en el Menú de Inicio de Windows, asegurando una organización estándar y localizable de las herramientas de desarrollo.



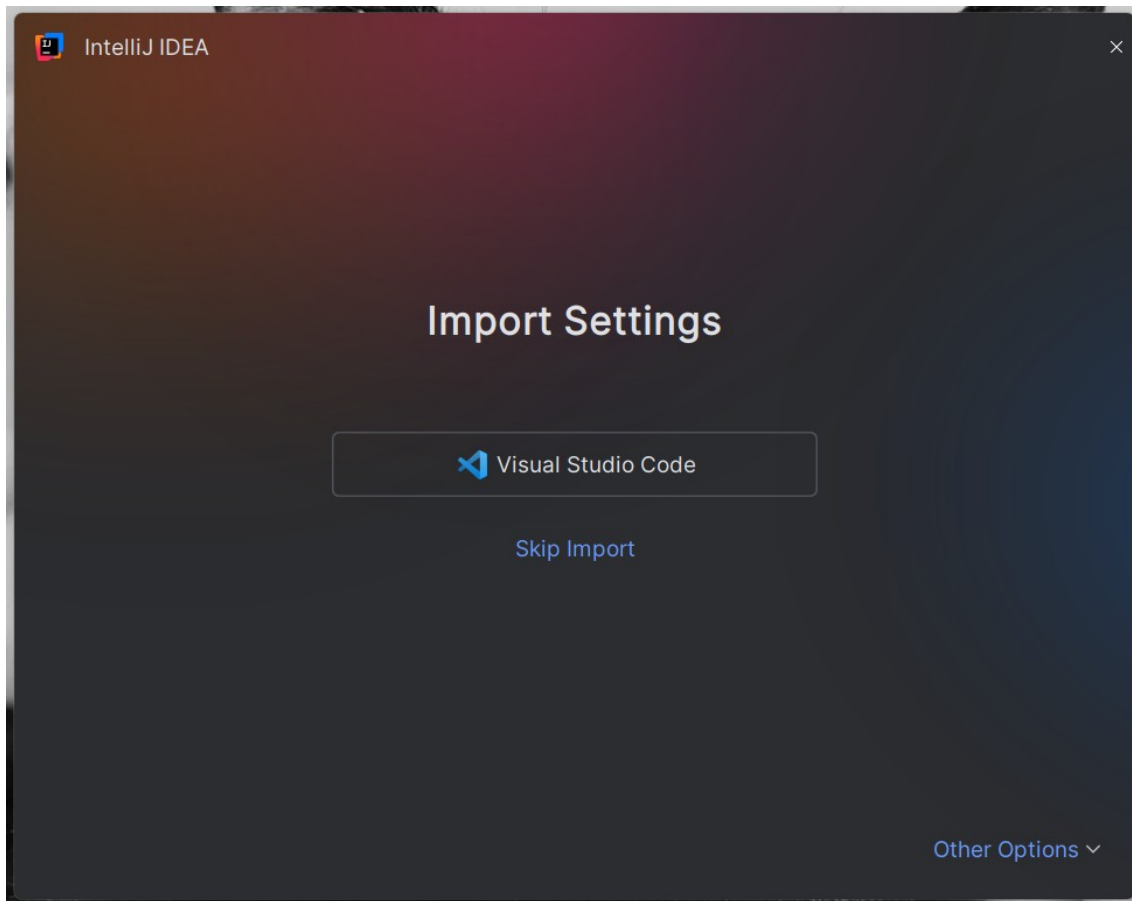
**Imagen 3:** 3.png (Finalización)

**Verificación de Instalación:** He completado el asistente marcando la opción "**Run IntelliJ IDEA**". Esto permite verificar inmediatamente que la instalación ha sido exitosa y que el software se ejecuta correctamente en el sistema operativo, cumpliendo el criterio de instalación.



**Imagen:** 4.png (Acuerdo de Licencia)

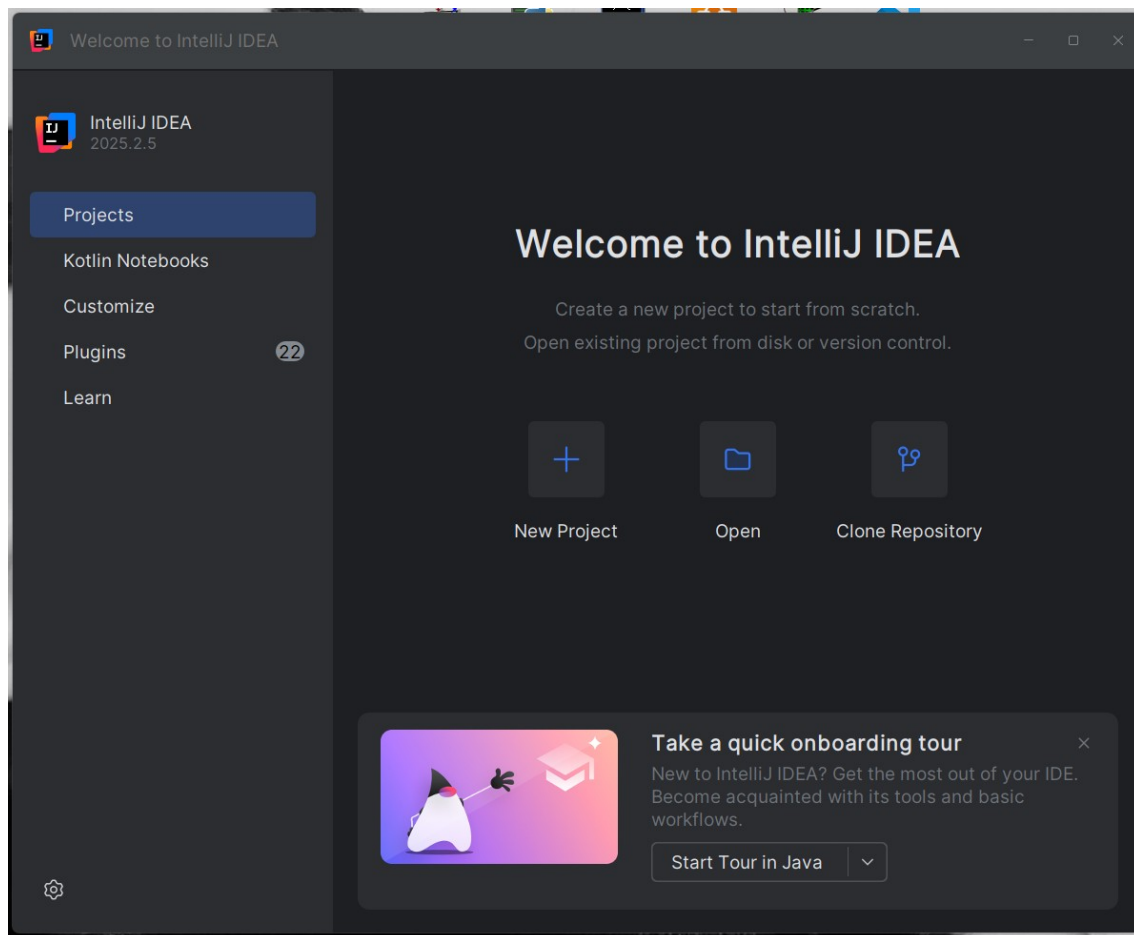
**Aceptación de Licencia de Software:** Como paso previo e imprescindible para la instalación del IDE propietario, he procedido a la lectura y aceptación del "User Agreement" de JetBrains. Este paso confirma la conformidad con los términos de uso, políticas de privacidad y protección de datos que rigen el uso de la versión Community de IntelliJ IDEA.



**Imagen:** 5.png (Import Settings)

**Decisión de Configuración Limpia (Clean Install):** Al iniciar el IDE por primera vez, el sistema detectó una instalación existente de Visual Studio Code y ofreció importar sus atajos y configuraciones. He seleccionado explícitamente **"Skip Import"** (Omitir importación).

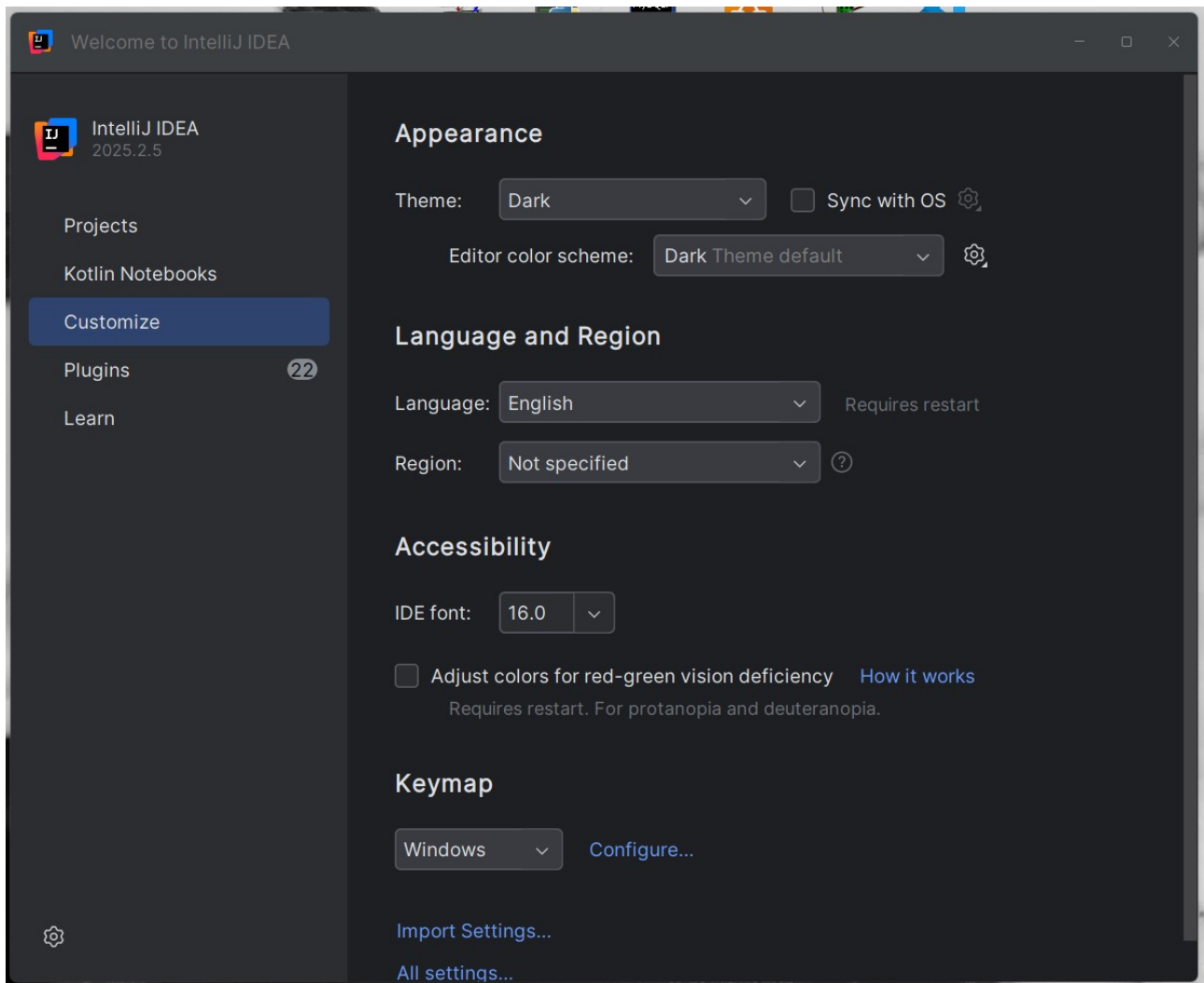
**Justificación:** Esta decisión es estratégica para la actividad, ya que me permite realizar una configuración y personalización manual desde cero (cumpliendo el criterio CE 2.c), evitando que el entorno propietario herede automáticamente preferencias que enmascararían el proceso de aprendizaje de sus menús nativos.



**Imagen 6:** 6.png (Pantalla de Bienvenida)

**Entorno Limpio:** Tras la ejecución inicial, se presenta la interfaz de bienvenida. He optado por no importar configuraciones previas para realizar una configuración manual y limpia ("Clean Install"), lo que me permitirá demostrar la personalización de módulos paso a paso.

## Configuración y Personalización



**Imagen 7:** 7.1.png (Apariencia)

**Personalización (CE 2.c):** He accedido al menú *Customize* para adaptar la ergonomía visual. He configurado el tema en "Dark" para reducir la fatiga visual y he aumentado el tamaño de la fuente del IDE a **16.0**, mejorando la legibilidad del código y la interfaz.

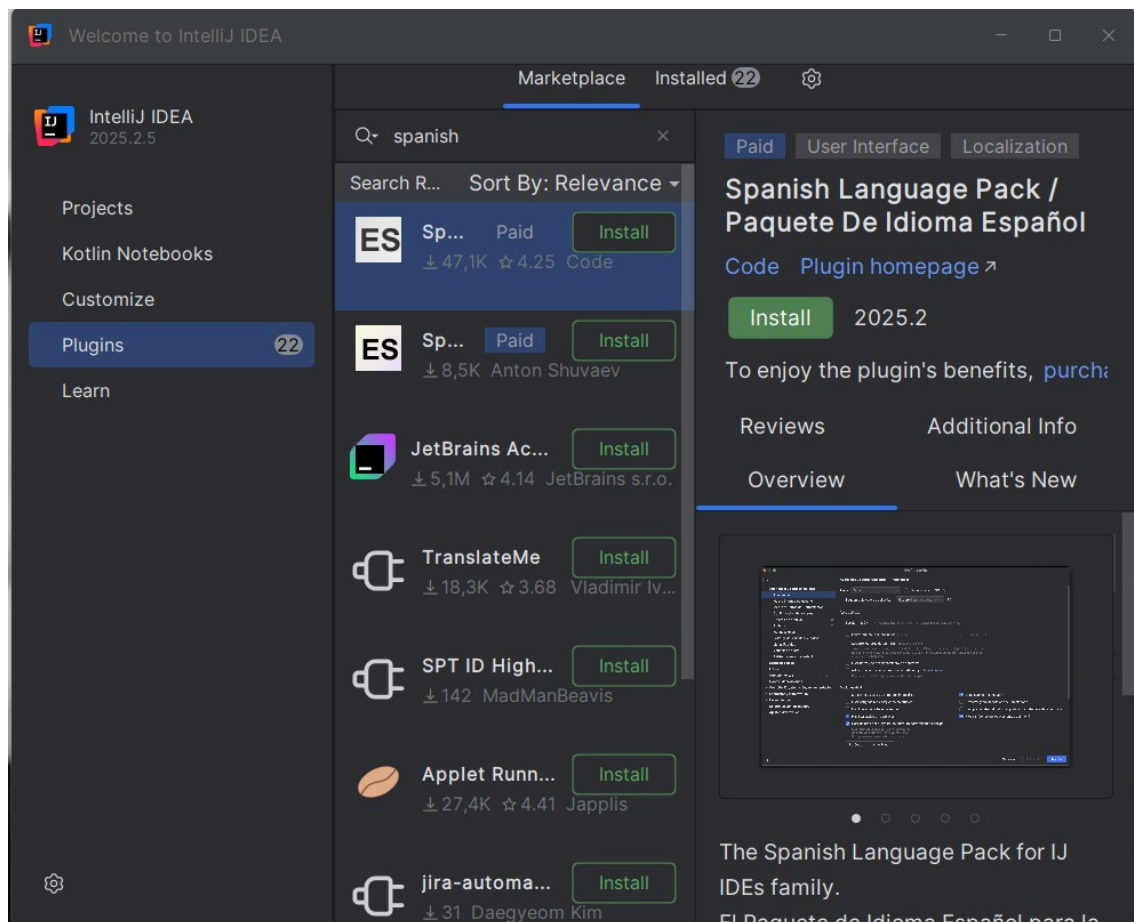
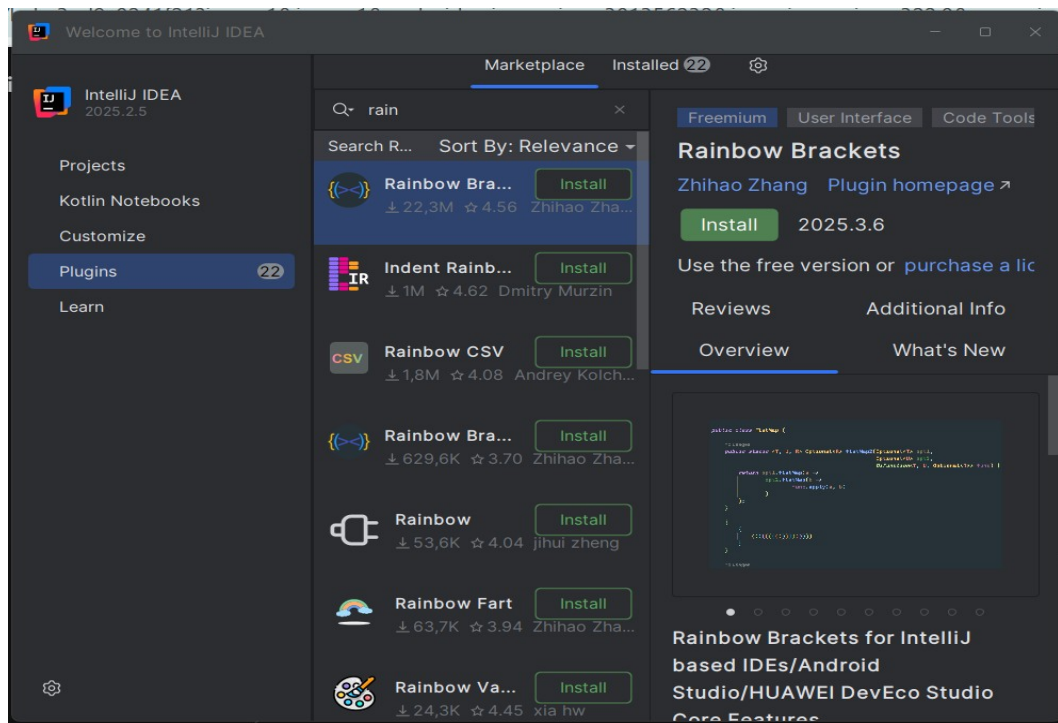
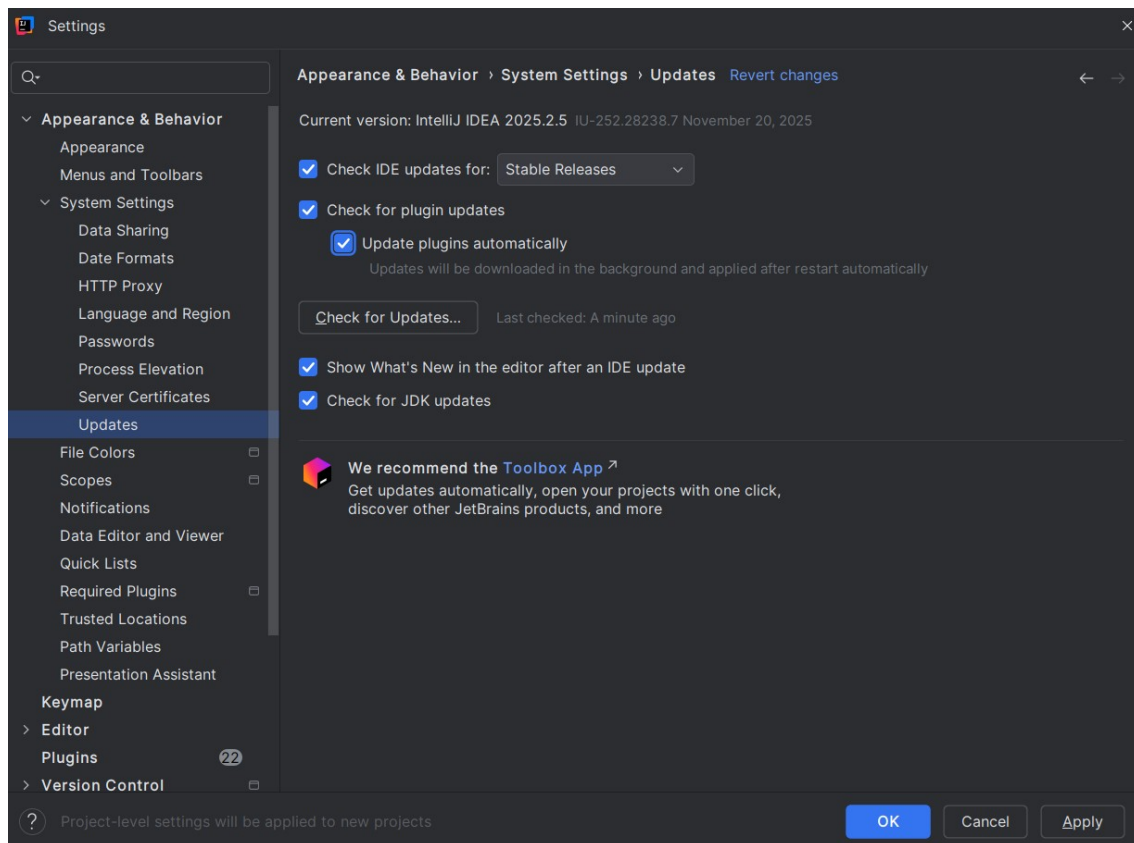


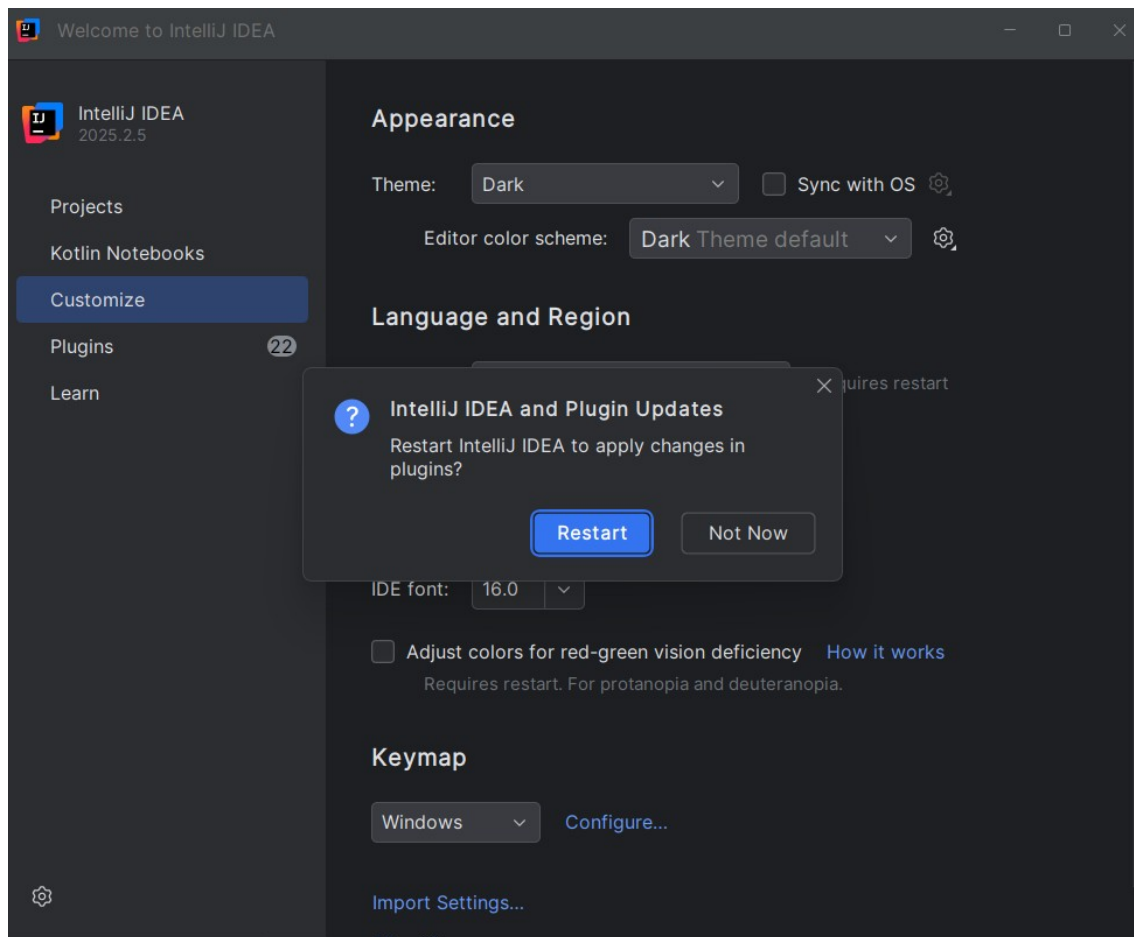
Imagen 8 y 9: 7.2.png y 7.3.png (Plugins)

**Gestión de Módulos (CE 2.b):** He extendido las funcionalidades base del IDE mediante el Marketplace. He instalado el plugin "**Rainbow Brackets**" para mejorar la lectura de bloques de código anidados y el "**Spanish Language Pack**" para la localización de la interfaz, demostrando la capacidad de gestión de extensiones del entorno.



**Imagen 10:** 7.5.png (Actualizaciones)

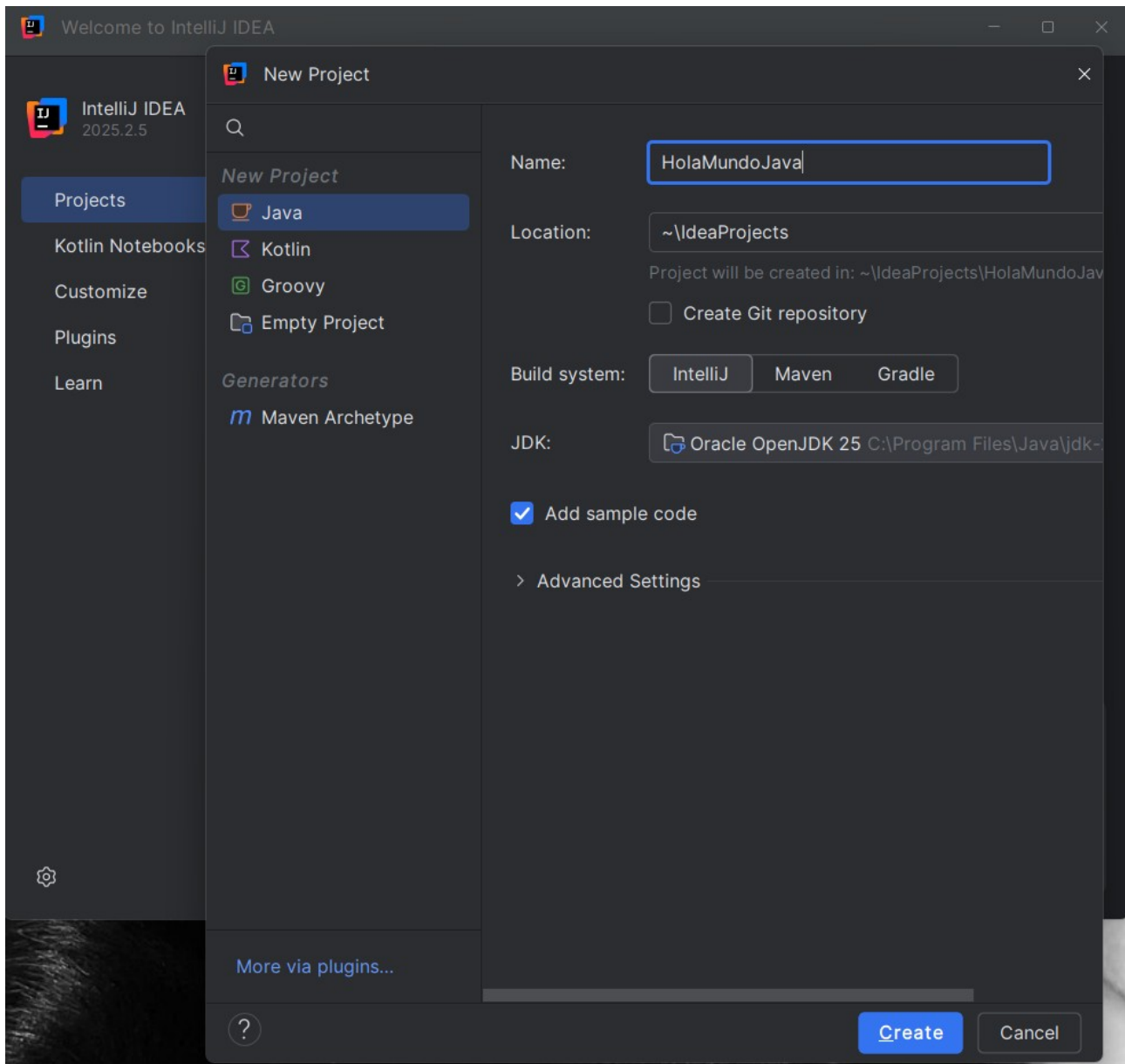
**Sistema de Actualizaciones (CE 2.d):** He verificado la configuración en *System Settings > Updates*. Como muestra la evidencia, he activado **"Check for plugin updates"** y **"Update plugins automatically"**, asegurando que el entorno se mantenga seguro y estable sin intervención manual.



**Imagen 11:** 7.4.png (Reinicio)

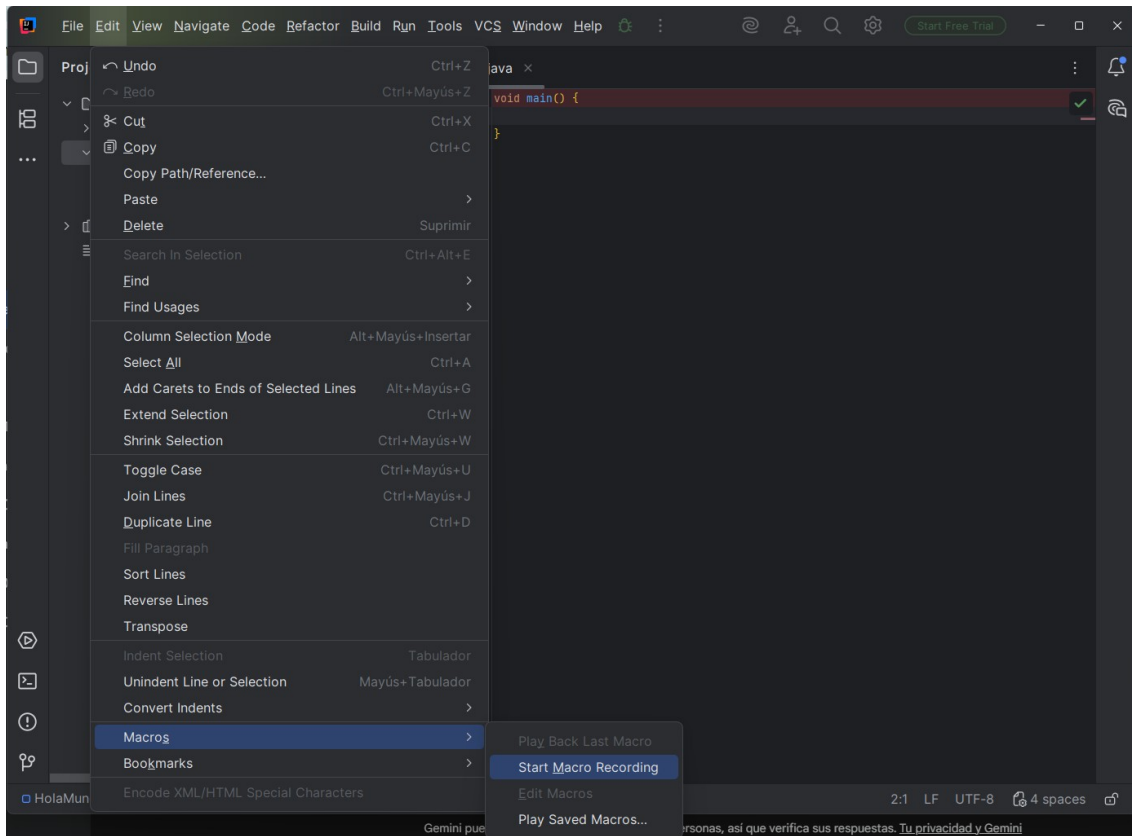
**Aplicación de Cambios:** El sistema gestiona las dependencias solicitando un reinicio ("Restart") para integrar los nuevos módulos en el núcleo ("Core") del IDE, confirmando la correcta instalación de los plugins.

## Automatización en IntelliJ



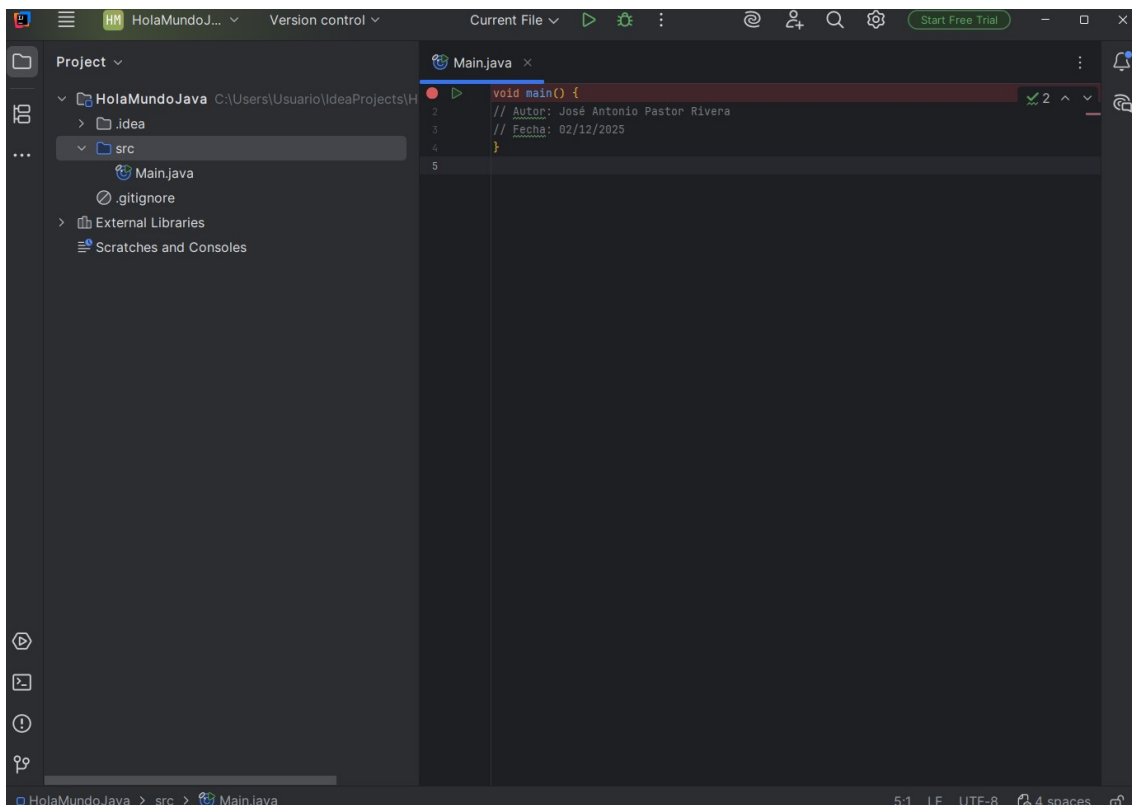
**Imagen 12:** 8.png (Crear Proyecto)

**Configuración del Proyecto:** He inicializado el proyecto "HolaMundoJava" seleccionando el **JDK 25 (Oracle OpenJDK)**. He configurado el sistema de construcción nativo de IntelliJ para agilizar el proceso de compilación en esta fase inicial.



**Imagen 13:** 9.png (Menú Macros)

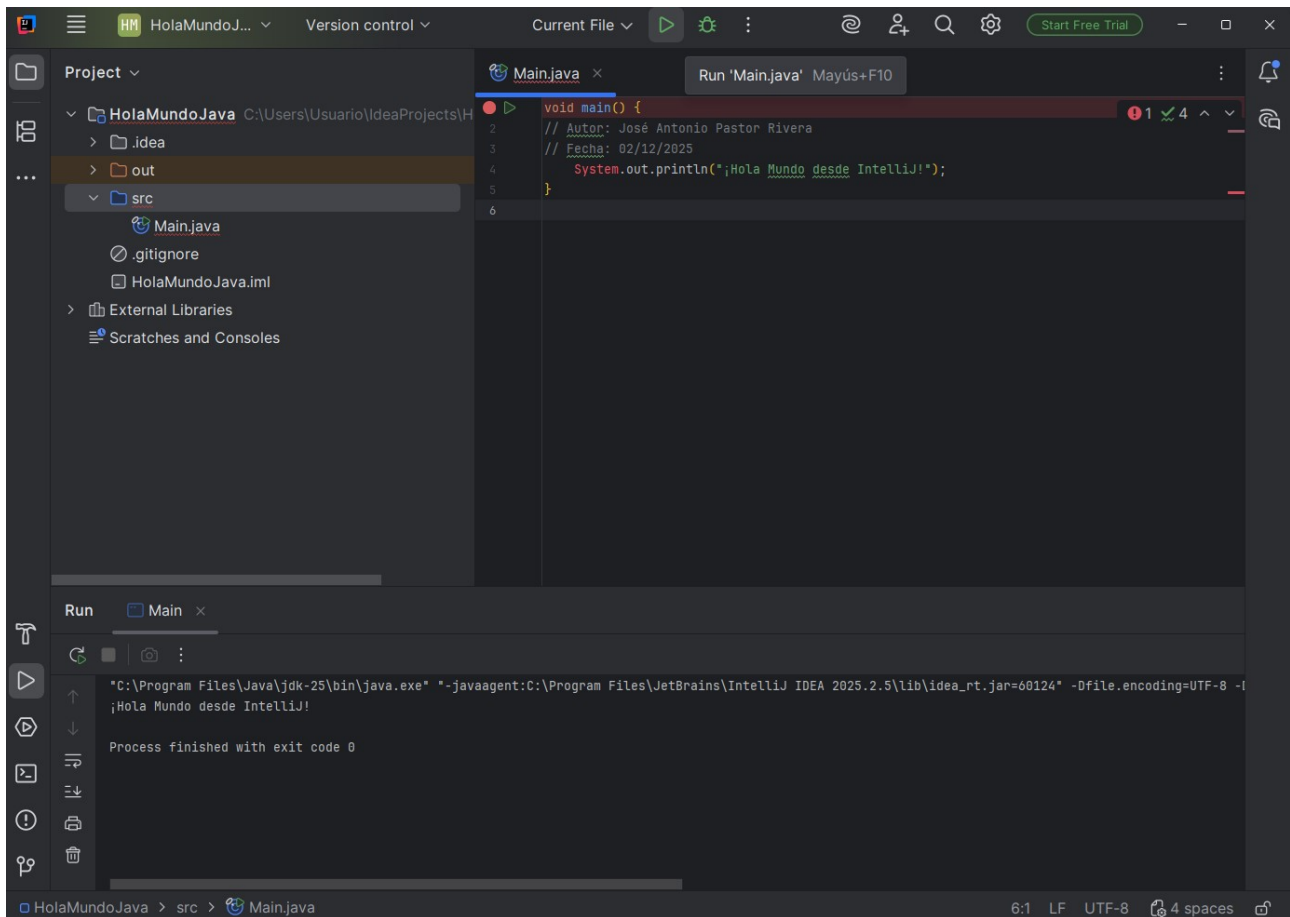
**Automatización (Grabación):** Para automatizar la firma de código, he utilizado la herramienta nativa de **Macros**. He accedido a *Edit > Macros > Start Macro Recording* para iniciar la captura de la secuencia de acciones repetitivas.



### Imagen 15: 10.png (Resultado Macro)

**Verificación Funcional:** La captura evidencia la ejecución exitosa de la macro. El bloque de comentarios se ha insertado automáticamente en el editor, validando que la herramienta de automatización es funcional y operativa.

### 4. Generación de Ejecutables – Java



### Imagen 16: 11.png (Ejecución Hola Mundo)

**Compilación y Ejecución (Java):** He implementado y ejecutado un código funcional en Java. La consola inferior muestra la salida *"¡Hola Mundo desde IntelliJ!"* y el código de salida 0, confirmando que el entorno compila y ejecuta correctamente el código fuente utilizando el JDK configurado.

## IDE Libre: Visual Studio Code

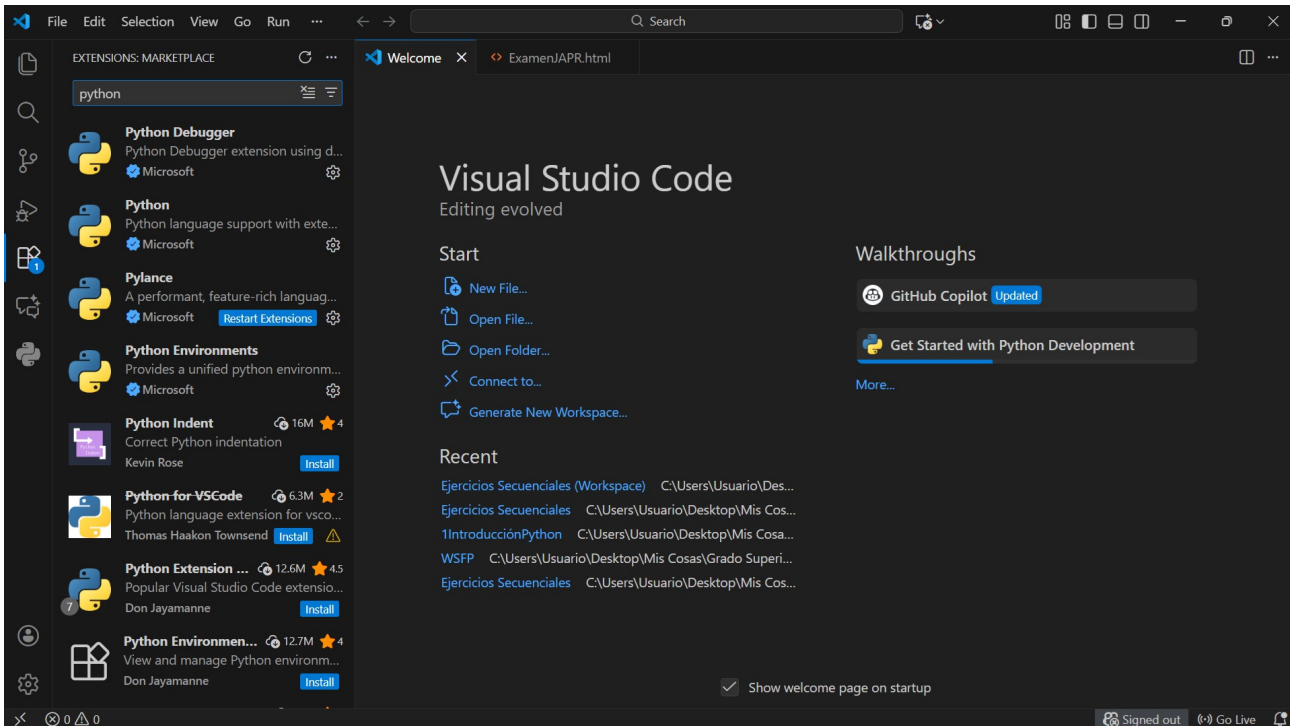


Imagen 17: 12.jpg (Extensiones VS Code)

**Gestión de Extensiones (IDE Libre):** En Visual Studio Code, he instalado la extensión oficial de **Python (Microsoft)** desde el Marketplace. Esto dota al editor de capacidades de IDE completo (Intellisense, Debugging) necesarias para el desarrollo multilenguaje.

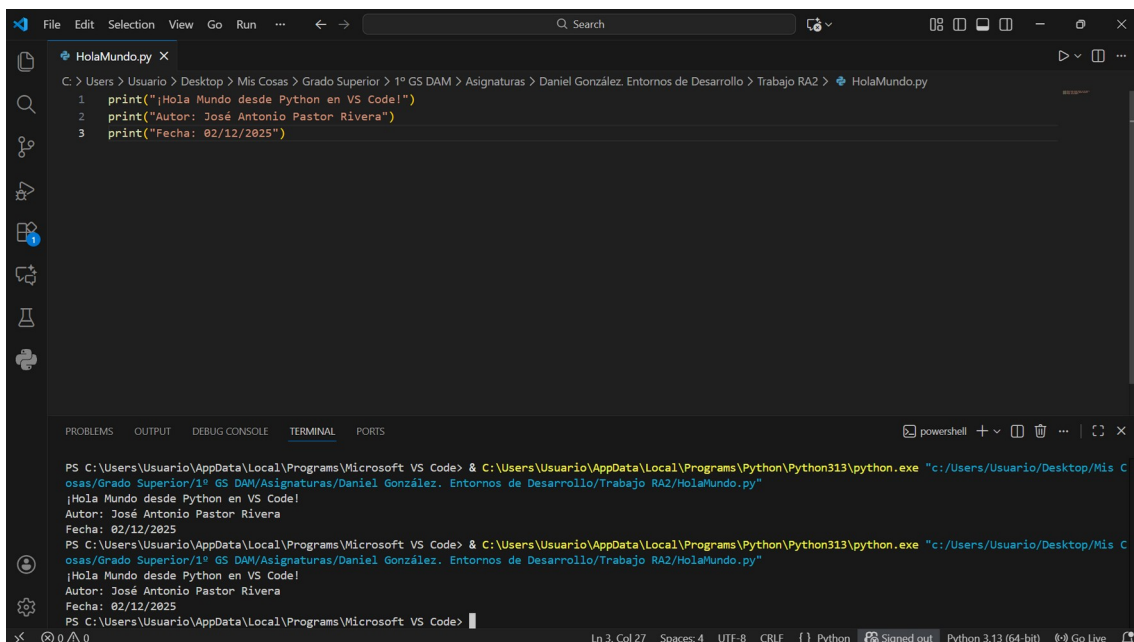


Imagen 18: 13.png (Ejecución Python)

**Ejecución Multilenguaje (Python):** He creado y ejecutado el script `HolaMundo.py`. La terminal integrada muestra la correcta interpretación del código Python, demostrando la flexibilidad del entorno libre para trabajar con tecnologías distintas a Java en la misma instancia.

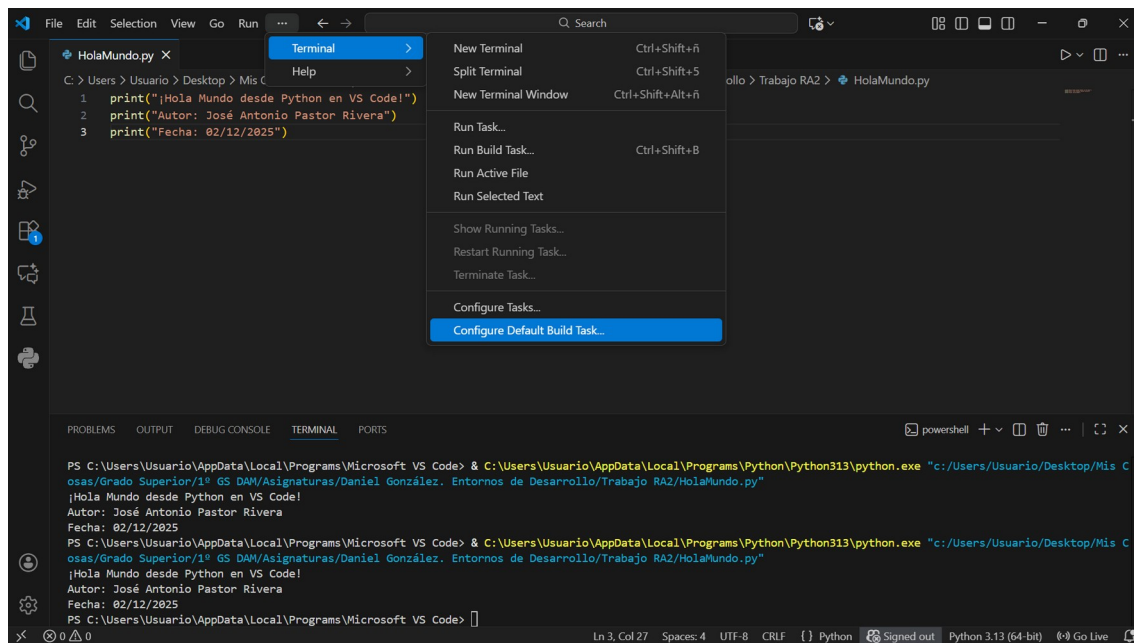


Imagen 19: 14.png

**Inicio de la Automatización (Configuración de Tareas):** Para implementar la automatización en el entorno libre (Visual Studio Code), he accedido al menú **Terminal > Configure Default Build Task**. A diferencia del sistema de grabación visual del IDE propietario, este paso es necesario para invocar el asistente que generará el archivo de definición `tasks.json`, donde se programará posteriormente la tarea automática.

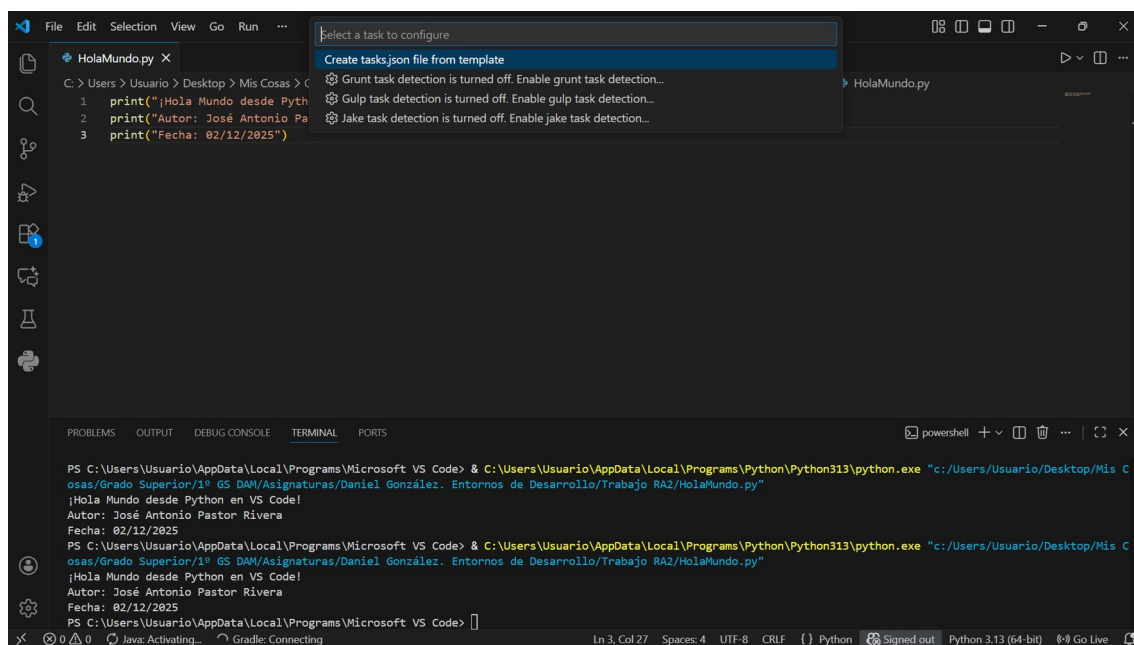


Imagen 20: 15.png

**Inicio de la Automatización (Configuración de Tareas):** Para implementar la automatización en el entorno libre (Visual Studio Code), he accedido al menú **Terminal > Configure Default Build Task**. Al no disponer de un grabador de macros visual nativo, he seleccionado la opción **"Create tasks.json file from template"**. Este paso es imprescindible para generar la estructura del archivo de configuración JSON donde definiremos la lógica de la tarea automatizada.

The image shows a screenshot of the Visual Studio Code editor. The top part of the window displays the `tasks.json` file, which is a JSON configuration for automated tasks. The file content is as follows:

```
1 // See https://go.microsoft.com/fwlink/?LinkId=733558
2 // for the documentation about the tasks.json format
3 "version": "2.0.0",
4 "tasks": [
5   {
6     "label": "echo",
7     "type": "shell",
8     "command": "echo Hello"
9   }
10 ]
```

The bottom part of the window shows the **TERMINAL** panel, which displays the output of the task execution. The terminal shows the command being run and the resulting output:

```
PS C:\Users\Usuario\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Usuario\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Usuario/Desktop/Mis C
osas/Grado Superior/1º GS DAM/Asignaturas/Daniel González. Entornos de Desarrollo/Trabajo RAZ/HolaMundo.py"
¡Hola Mundo desde Python en VS Code!
Autor: José Antonio Pastor Rivera
Fecha: 02/12/2025
PS C:\Users\Usuario\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Usuario\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Usuario/Desktop/Mis C
osas/Grado Superior/1º GS DAM/Asignaturas/Daniel González. Entornos de Desarrollo/Trabajo RAZ/HolaMundo.py"
¡Hola Mundo desde Python en VS Code!
Autor: José Antonio Pastor Rivera
Fecha: 02/12/2025
PS C:\Users\Usuario\AppData\Local\Programs\Microsoft VS Code>
```

**Imagen 21:** 16.png

**Definición y Ejecución de la Tarea Automatizada:** He editado el archivo de configuración `tasks.json` para definir la lógica de la automatización. He creado una tarea etiquetada como "echo", de tipo shell, que ejecuta un comando de sistema para imprimir mis datos y la fecha.

Como se evidencia en la terminal inferior, tras lanzar la tarea ("Run Build Task"), Visual Studio Code interpreta correctamente el JSON y ejecuta la instrucción, validando el funcionamiento de la automatización mediante scripts de configuración.

The screenshot shows the Visual Studio Code editor with a file named `tasks.json` open. The file contains a JSON configuration for a task named "FirmaAutomatica". The configuration includes a version, a task label, a shell type, a command to echo a specific message with a date, and a group configuration for building. Below the editor, the terminal shows the execution of this task, displaying the echoed message. The status bar at the bottom indicates the current line and column, as well as the file encoding and format.

```
1 {
2   "version": "2.0.0",
3   "tasks": [
4     {
5       "label": "FirmaAutomatica",
6       "type": "shell",
7       "command": "echo 'Tarea ejecutada por: José Antonio Pastor - Fecha: 02/12/2025'",
8       "group": {
9         "kind": "build",
10        "isDefault": true
11      }
12    }
13  ]
14 }
```

Executing task: echo 'Tarea ejecutada por: José Antonio Pastor - Fecha: 02/12/2025'

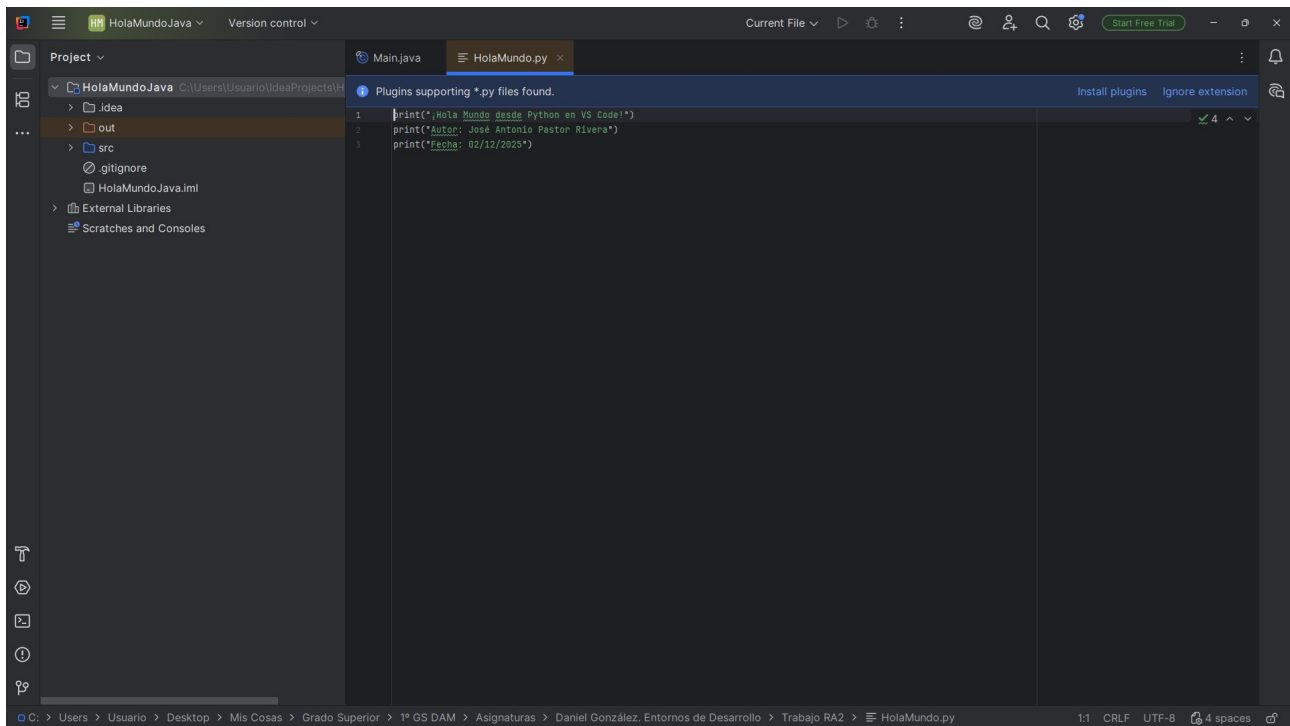
Tarea ejecutada por: José Antonio Pastor - Fecha: 02/12/2025

Terminal will be reused by tasks, press any key to close it.

**Imagen 20:** 17.png

**Automatización Avanzada (Tasks):** Dado que VS Code utiliza un enfoque basado en configuración, he implementado la automatización mediante el archivo `tasks.json`. He definido una tarea personalizada ("FirmaAutomatica") que ejecuta comandos de sistema, demostrando un nivel avanzado de configuración manual en entornos libres.

## Interoperabilidad



**Imagen 16:** 18.png (Aviso Plugin Python en IntelliJ)

**Interoperabilidad y Detección de Contexto:** Al abrir el archivo `HolaMundo.py` (creado en VS Code) dentro de IntelliJ, el IDE propietario ha demostrado su inteligencia contextual. Al detectar un archivo Python sin tener el soporte activado, el sistema ha sugerido proactivamente ("Plugins supporting \*.py files found") la instalación de las herramientas necesarias, facilitando la migración y compatibilidad entre entornos.

### Reflexión y Comparativa de Entornos

Tras completar el ciclo de instalación, configuración, automatización y ejecución en ambos entornos, presento el análisis comparativo entre **IntelliJ IDEA** (IDE Propietario) y **Visual Studio Code** (IDE Libre).

1. ¿Qué características comunes has encontrado en ambos?

A pesar de sus diferentes arquitecturas, he identificado una base funcional compartida que define a un entorno de desarrollo moderno:

**Modularidad:** Ambos basan gran parte de su potencia en un ecosistema de extensiones. Tanto el "Marketplace" de VS Code como el gestor de "Plugins" de IntelliJ permiten añadir soporte para lenguajes (como Python) o herramientas visuales (temas, iconos).

**Terminal Integrada:** En ambos casos he podido ejecutar código y scripts directamente desde una consola empotrada en la interfaz, sin necesidad de salir al sistema operativo.

**Sistemas de Automatización:** Ambos permiten automatizar tareas repetitivas, aunque la implementación difiere (visual vs. textual).

**Gestión de Proyectos:** Ambos estructuran el trabajo en carpetas o proyectos, permitiendo la navegación por la jerarquía de ficheros en una barra lateral.

2. ¿Qué características específicas te han llamado la atención de cada uno?

**En IntelliJ IDEA (Propietario):** Me ha sorprendido su capacidad de "**Conciencia de Contexto**".

Un ejemplo claro fue durante la prueba de interoperabilidad: al abrir un archivo .py sin tener Python configurado, el IDE no solo mostró el texto, sino que analizó la extensión y me sugirió proactivamente instalar el plugin necesario mediante una notificación inteligente. Además, su sistema de **Macros** es nativo y visual (Grabar/Parar), lo que lo hace muy accesible.

**En Visual Studio Code (Libre):** Destaca su enfoque de "**Editor Ligero**". No te obliga a crear una estructura de proyecto compleja para ejecutar un simple script; puedes abrir un archivo suelto y ejecutarlo. Su característica específica más notable es que la configuración se basa en archivos de texto (JSON). Por ejemplo, para la automatización no usé una grabadora visual, sino que tuve que configurar explícitamente el archivo tasks.json, lo cual ofrece un control más granular a cambio de una mayor curva de aprendizaje.

3. ¿Cuál te ha resultado más cómodo, y por qué?

Para tareas rápidas y edición de scripts sencillos, **Visual Studio Code** me ha resultado más cómodo. Su interfaz es más limpia por defecto y el tiempo de carga es perceptiblemente menor. La sensación de fluidez al escribir código en múltiples lenguajes sin tener que esperar a que se "indexe" todo un proyecto pesado hace que la experiencia de usuario sea muy ágil.

4. ¿Cuál te ha parecido más potente, y por qué?

Sin duda, **IntelliJ IDEA** me ha parecido la herramienta más potente y robusta, especialmente para el desarrollo en Java. Mientras que en VS Code tuve que "construir" mi entorno pieza a pieza, IntelliJ ofrece una experiencia "**todo incluido**". Herramientas como el gestor de SDKs (que me permitió descargar el JDK 25 desde la propia interfaz), la grabación de macros integrada en el menú y su capacidad de refactorización y análisis de código demuestran que es un software diseñado para ingeniería de software profesional y proyectos de gran envergadura, donde la asistencia inteligente justifica su mayor consumo de recursos.